Złożoność algorytmiczna

Teoria złożoności obliczeniowej – jej celem jest oszacowanie ile zasobów jest potrzebnych do rozwiązania problemu obliczeniowego.

Zasoby:
- czas – złożoność czasowa,
- pamięć – złożoność pamięciowa.

Złożoność algorytmu jest funkcją rozmiaru danych wejściowych.

Złożoność może być:
- pesymistyczna – najgorszego przypadku,
- średnia.

Złożoność algorytmiczna

Algorytmy porównuje się biorąc pod uwagę asymptotyczne tempo wzrostu.

Asymptotyczne tempo wzrostu opisuje zachowanie (górną granicę) funkcji gdy jej argumenty dążą do nieskończoności.

Notacja Duże – O (Big – O).

Big – O

Niech dane będą dwie funkcje $f(x)$ i $g(x)$. Wtedy

$$f(x) = O\big(g(x)\big) \text{ dla } x \to \infty$$

wtedy i tylko wtedy gdy istnieje taka dodatnia stała M i liczba rzeczywista x_0, że

$$|f(x)| \leq M|g(x)| \text{ dla wszystkich } x \geq x_0$$

Big – O, przykład

$$f(x) = 3x^4 + 2x + 10000$$

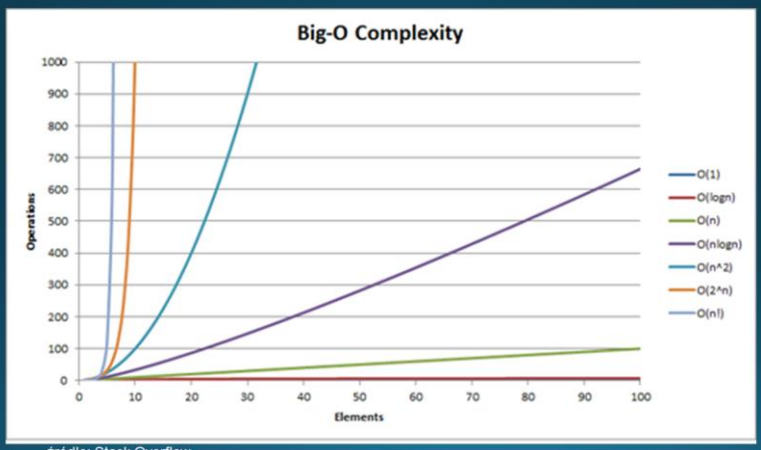
dla małych x dominuje w wyniku 10000
dla rosnących x zaczyna dominować składnik $3x^4$
dla bardzo dużych x wpływ na wynik ma jedynie składnik x^4

Złożoność przykładowej funkcji wynosi więc $O(n^4)$

Rodzaje złożoności

- stała $O(1)$,
- logarytmiczna $O(\log n)$,
- liniowa $O(n)$,
- kwadratowa $O(n^2)$,
- wielomianowa $O(n^c)$,
- wykładnicza $O(c^n)$

Rodzaje złożoności



źródło: Stack Overflow

Big - O

Czas wymagany do realizacji algorytmu zależy proporcjonalnie od liczby operacji podstawowych:

- operacje arytmetyczne,
- przypisanie,
- porównanie,
- odczytanie wartości,
- zapisanie wartości,
- itp..

Big - O

Sekwencja wyrażeń:

wyrażenie_1

wyrażenie_2

…

wyrażenie_n

czas_wykonania = czas(wyrażenie_1) + czas(wyrażenie_2) + … + czas(wyrażenie_n)

Jeśli wyrażenie obejmują operacje podstawowe, to ich złożoność wyności O(1). Złożoność sekwencji również wynosi wtedy O(1)

Big - O

```
if - else:
        if (warunek)
                blok_1
        else
                blok_2
```

czas_wykonania = max(czas(blok_1), czas(blok_2))

Big - O

Pętle, np. for

```
for (int i = 0; i < N, i++)
        blok
```

Jeśli złożoność bloku jest liniowa – O(1) to złożoność pętli wynosi N*O(1) czyli O(N).

Big - O

Pętle zagnieżdżone, np. for

```
for (int j = 0; j < M; j++)
        for (int i = 0; i < N, i++)
                blok
```

Jeśli złożoność bloku jest liniowa – O(1) to złożoność pętli wynosi N*M*O(1) czyli O(N*M). Jeśli M == N to złożoność wynosi $O(N^2)$

Sortowanie

1. Sortowanie przez wstawiania (ang. insertion sort),
2. Sortowanie przez wybieranie (ang. selection sort),
3. Sortowanie bąbelkowe (ang. bubble sort),
4. Metoda Shella (ang. shellsort),
5. Kopcowanie (ang. heapsort),
6. Sortowanie szybkie (ang. quicksort),
7. Sortowanie przez scalanie (ang. mergesort).

Sortowanie przez wstawianie

Insertion sort:

15, 2, 11, 7, 3, 1, 12, 19, 17
15, 2, 11, 7, 3, 1, 12, 19, 17
2, 15, 11, 7, 3, 1, 12, 19, 17
2, 15, 11, 7, 3, 1, 12, 19, 17
2, 11, 15, 7, 3, 1, 12, 19, 17
2, 11, 15, 7, 3, 1, 12, 19, 17
2, 7, 11, 15, 3, 1, 12, 19, 17

Sortowanie przez wstawianie

Insertion sort:

2, 7, 11, 15, 3, 1, 12, 19, 17
2, 3, 7, 11, 15, 1, 12, 19, 17
2, 3, 7, 11, 15, 1, 12, 19, 17
1, 2, 3, 7, 11, 15, 12, 19, 17
1, 2, 3, 7, 11, 15, 12, 19, 17
1, 2, 3, 7, 11, 12, 15, 19, 17
1, 2, 3, 7, 11, 12, 15, 19, 17
1, 2, 3, 7, 11, 12, 15, 19, 17

Sortowanie przez wstawianie

Insertion sort:

1, 2, 3, 7, 11, 12, 15, 19, 17

1, 2, 3, 7, 11, 12, 15, 17, 19

Sortowanie przez wstawianie

Realizacja wstawiania:

2, 11, 15, 7, 3, 1, 12, 19, 17

2, 11, 7, 15, 3, 1, 12, 19, 17

2 , 7, 11, 15, 3, 1, 12, 19, 17

Złożoność obliczeniowa $O(n^2)$

Sortowanie przez wstawianie

Kroki (sortowanie rosnące):

1. Z nieposortowanego zbioru liczb weź pierwszą liczbę X,
2. Rozpoczynając od końca posortowanego zbioru porównuj liczbę X z kolejnymi liczbami Y zbioru posortowanego,
3. Jeśli liczba X jest mniejsza od Y zamień je miejscami,
4. Jeśli warunek z punktu 3 nie jest spełniony lub koniec zbioru posortowanego zakończ porównywanie,
5. Jeśli zbiór nieposortowany jest niepusty wróć do punktu nr 1.

Sortowanie przez wybieranie

Selection sort:

15, 2, 11, 7, 3, 1, 12, 19, 17

1, 2, 11, 7, 3, 15, 12, 19, 17

1, 2, 11, 7, 3, 15, 12, 19, 17

1, 2, 11, 7, 3, 15, 12, 19, 17

1, 2, 11, 7, 3, 15, 12, 19, 17

1, 2, 3, 7, 11, 15, 12, 19, 17

1, 2, 3, 7, 11, 15, 12, 19, 17

Sortowanie przez wybieranie

1, 2, 3, 7, 11, 15, 12, 19, 17
1, 2, 3, 7, 11, 15, 12, 19, 17
1, 2, 3, 7, 11, 15, 12, 19, 17
1, 2, 3, 7, 11, 15, 12, 19, 17
1, 2, 3, 7, 11, 12, 15, 19, 17
1, 2, 3, 7, 11, 12, 15, 19, 17
1, 2, 3, 7, 11, 12, 15, 19, 17
1, 2, 3, 7, 11, 12, 15, 17, 19
1, 2, 3, 7, 11, 12, 15, 17, 19
1, 2, 3, 7, 11, 12, 15, 17, 19

Złożoność obliczeniowa $O(n^2)$

Sortowanie przez wybieranie

Kroki (sortowanie rosnące):

1. Z nieposortowanego wybierz najmniejszą wartość X,
2. Zamień wartość X z pierwszą wartością zbioru nieposortowanego,
3. Jeśli zbiór nieposortowany jest niepusty wróć do punktu nr 1.

Sortowanie bąbelkowe

Bubble sort:

15, 2, 11, 7, 3, 1, 12, 19, 17
2, 15, 11, 7, 3, 1, 12, 19, 17
2, 15, 11, 7, 3, 1, 12, 19, 17
2, 11, 15, 7, 3, 1, 12, 19, 17
2, 11, 15, 7, 3, 1, 12, 19, 17
2, 11, 7, 15, 3, 1, 12, 19, 17
2, 11, 7, 15, 3, 1, 12, 19, 17

Sortowanie bąbelkowe

2, 11, 7, 15, 3, 1, 12, 19, 17

2, 11, 7, 3, 15, 1, 12, 19, 17

2, 11, 7, 3, 15, 1, 12, 19, 17

2, 11, 7, 3, 1, 15, 12, 19, 17

2, 11, 7, 3, 1, 15, 12, 19, 17

2, 11, 7, 3, 1, 12, 15, 19, 17

2, 11, 7, 3, 1, 12, 15, 19, 17

2, 11, 7, 3, 1, 12, 15, 17, 19

Sortowanie bąbelkowe

Kroki (sortowanie rosnące):

1. Od początku do końca zbioru nieposortowanego porównuj dwie sąsiednie wartości,
2. Jeśli „lewa” wartość jest większa od „prawej” zamień je miejscami,
3. Jeśli warunek 2 nie jest spełniony przejdź dalej,
4. Jeśli zbiór nieposortowany jest niepusty wróć do punktu nr 1.

Sortowanie można zakończyć wcześniej jeśli w danej iteracji nie dokonano żadnej zamiany wartości.

Złożoność obliczeniowa $O(n^2)$

Sortowanie metodą Shella

Shellsort:

9, 2, 11, 13, 3, 1, 12, 19, 17 – posortuj elementy odległe o 5
1, 2, 11, 13, 3, 9, 12, 19, 17

1, 2, 11, 13, 3, 9, 12, 19, 17 – posortuj elementy odległe o 3
1, 2, 9, 12, 3, 11, 13, 19, 17

1, 2, 9, 12, 3, 11, 13, 19, 17 – posortuj elementy odległe o 1

Sortowanie metodą Shella

Sortowanie Shella jest modyfikacją sortowania przez wstawianie.

Odstępy mogą się zmieniać zgodnie z formułą:

$$2^k + 1$$

Inne odstępy są również możliwe. Wybór zbioru odstępów decyduje o złożoności obliczeniowej.

Złożoność obliczeniowa dla powyższych odstępów: $O(n^{1,5})$